

AI Agent 엔지니어링

프로덕션에서 동작하는 신뢰할 수 있는 AI 에이전트 설계와 구현

AI Agent 엔지니어링

프로덕션에서 동작하는 신뢰할 수 있는 AI 에이전트 설계와 구현

ThakiCloud (Hyojung Han)

Contents

1	도구 호출 계약: 에이전트와 외부 세계의 접점	1
2	리트라이와 에러 복구 루프: 실패를 견디는 시스템	5
3	Observability: 에이전트 내부에서 무슨 일이 일어나는지 알기	9
4	인간-에이전트 협업 모델: 자동화와 통제의 경계	13

1 도구 호출 계약: 에이전트와 외부 세계의 접점

AI 에이전트가 프로덕션에서 안정적으로 동작하려면 도구 호출 계약이 명확해야 한다. 에이전트가 외부 시스템과 데이터를 주고받는 방식, 즉 도구 스키마의 설계가 전체 시스템의 신뢰성을 좌우한다. 이 장에서는 도구 스키마를 API 계약으로 다루면서 실패를 최소화하는 실전 설계 방법을 정리한다.

1.1 도구 스키마는 버전 관리 대상 계약이다

웹 API를 설계할 때는 스키마 변경을 신중하게 다룬다. AI 에이전트의 도구 스키마도 마찬가지다. 도구 설명의 단어 하나가 에이전트의 판단을 바꾸고 출력 형식이 바뀌면 후속 처리가 통째로 무너진다. 도구 스키마를 느슨한 JSON으로 넘겨서는 안 되는 이유다.

좋은 도구 스키마가 명확히 해야 할 것은 네 가지다. 먼저 각 파라미터의 이름과 타입을 엄격히 명시해야 한다. `user_id`가 문자열인지 정수인지 모호하면 에이전트가 가끔 정수를 보내고 API가 500을 반환하는 사태가 생긴다. 필수 파라미터와 선택 파라미터의 구분도 필요하다. 에이전트가 선택 파라미터를 매번 채우는 건 낭비지만 필수 파라미터를 빠뜨리면 실행 자체가 실패한다. 출력 구조는 문서화해야 도구가 무엇을 반환하는지 에이전트가 알고 그 결과를 제대로 활용할 수 있다. 마지막으로 에러 시나리오를 명시한다. 어떤 입력이 어떤 에러를 부르는지 알려주면 에이전트가 재시도할지 대체 동작으로 넘어갈지 스스로 판단한다.

스키마를 설계한 후에는 샘플 입출력 쌍으로 검증한다. 에이전트가 실제로 어떻게 호출하는지 지켜보고 스키마와 실행 사이의 간극을 확인한다.

1.2 타입과 enum으로 출력 포맷 정규화하기

에이전트는 텍스트 생성기라서 도구 출력의 형식을 매번 다르게 만들어낼 수 있다. 한 번은 `{"result": "ok", "data": [...]}` 형태로, 다음에는 `{"status": "success", "items": [...]}` 형태로 나온다. 이러면 후속 처리를 맡은 에이전트가 파싱에서 실수한다.

해결책은 출력 포맷을 정규화하는 것이다. 모든 도구가 같은 응답 구조를 따르면 에이전트의 행동도 예측 가능해진다.

응답 구조는 세 가지 필드로 구성한다. `success`는 불린으로 성공 여부를 표시한다. `data`는 실제 반환값이고 실패 시엔 `null`을 넣는다. `error`는 에러 메시지 문자열이며 성공 시엔 `null`을 넣는다. 이 구조를 모든 도구가 따르면 에이전트는 `success`가 `true`일 때만 `data`를 사용하면 된다.

Enum 타입의 활용도 중요하다. 상태값이 정해진 집합이라면 문자열보다 `enum`을 쓴다. `"status": "pending" | "processing" | "done" | "failed"`로 제한하면 에이전트가 유효하지 않은 값을 넣어볼 가능성을 원천 차단한다.

1.3 도구 설명이 에이전트의 판단을 이끈다

도구 설명은 단순한 문서가 아니라 에이전트에게 주는 지시다. 어떻게 설명하느냐에 따라 에이전트가 도구를 선택하는 방식과 호출 시 전달하는 인자가 달라진다.

좋은 도구 설명은 한 줄 요약으로 시작해 각 파라미터의 의미를 설명하고 사용 예시를 포함한다. 특히 경계 조건을 명시하는 것이 중요하다. “이 파라미터에 `null`을 넣으면 전체 조화가 됩니다”처럼 에이전트가 헛갈릴 수 있는 상황은 미리 알려준다.

도구 설명을 작성한 후에는 에이전트에게 직접 물어보는 방법이 있다. “이

도구를 언제 호출하면 안 되는가?”라는 질문에 에이전트가 제대로 답하면 설명이 충분하다는 뜻이다. 답이 불안하면 설명을 보완한다.

1.4 실전: 데이터베이스 조회 도구 스키마 설계

사용자를 조회하는 도구를 만든다고 하자. 나쁜 스키마는 파라미터 없이 결과를 통째로 반환한다. 에이전트가 수만 건을 한 번에 처리하려다 메모리가 터진다.

좋은 스키마는 페이지네이션을 필수로 하며 필터 조건을 명시적으로 지원한다. `limit`과 `offset`으로 결과를 분할하고 `created_after`와 `status` 같은 필터 파라미터를 제공한다. 에이전트가 처음 조회할 때 `limit` 기본값을 100으로 설정하고, 필요하면 `offset`으로 페이지를 넘기도록 유도한다.

결과 스키마에도 페이지네이션 메타데이터를 포함한다. `total_count`, `has_more`, `next_offset` 같은 필드를 추가하면 에이전트가 추가 조회가 필요한지 스스로 판단할 수 있다.

1.5 계약의 진화: 하위 호환성 관리

도구 스키마는 시간이 지나면 변한다. 새로운 파라미터 추가, 출력 필드 변경, `deprecated` 필드 처리가 필요하다. 이때 기존에 동작하던 에이전트가 갑자기 실패하면 시스템 전체에 영향을 준다.

하위 호환성을 지키는 원칙은 세 가지로 요약된다. 기존 파라미터는 삭제하지 않고 `deprecated` 표시만 남겨 응답에 그대로 포함한다. 새로운 파라미터는 선택적으로 만들어야 한다. 필수로 만들면 기존 에이전트가 전부 깨진다. 출력 필드도 절대 삭제하지 않는다. 대신 `null`을 반환하거나 `deprecated` 주석을 붙인다.

도구 스키마의 변경 이력을 관리하면 디버깅이 쉬워진다. 각 버전의 스키마

문서를 유지하고, 에이전트가 특정 버전의 스키마를 요청하면 해당 버전의 행동을 보장한다.

1.6 정리

도구 호출 계약은 에이전트 시스템의 기반이다. 스키마를 엄격하게 정의하고 출력 포맷을 정규화하며 도구 설명을 충실히 쓰고 하위 호환성까지 관리하면 런타임 실패를 크게 줄일 수 있다.

2 리트라이와 에러 복구 루프: 실패를 견디는 시스템

네트워크는 언제나 실패한다. API 서버가 잠시 응답하지 않고, 데이터베이스 연결이 끊어지고, 레디스가 타임아웃된다. 에이전트가 이런 상황에서 바로 멈추면 프로덕션 시스템은 하루에 수십 번 중단된다. 실패를 견디는 시스템을 만들려면 리트라이 정책과 에러 복구 루프를 설계해야 한다.

2.1 재시도가 필요 없는 오류와 필요한 오류를 구분하라

모든 에러를 똑같이 취급하면 안 된다. 에러는 크게 세 가지로 나뉜다.

일시적 에러는 시간이 지나면 사라진다. 네트워크 타임아웃이나 데이터베이스 잠금, 서버 과부하가 여기 속하며 재시도로 해결할 수 있다. 반면 영구적 에러는 몇 번을 재시도해도 같은 결과가 돌아온다. 잘못된 파라미터, 존재하지 않는 리소스 조회, 권한 부족이 대표적이고 재시도할수록 시스템에 부담만 늘어난다. 파사드 에러는 원래 에러를 다른 형태로 감싸서 반환하는 경우다. gRPC 상태 코드를 HTTP 상태 코드로 바꾸는 과정에서 원래 에러 정보가 사라지는 게 그 예다.

이렇게 구분해두면 에이전트의 판단 비용이 줄어든다. 에러 타입을 에이전트가 직접 가려내려 하면 매번 복잡한 로직이 필요하지만, 시스템이 세 유형을 미리 나눠주면 에이전트는 단순한 규칙만 적용하면 된다.

2.2 지수적 백오프와 jitter

단순 무한 재시도는 결국 네트워크가 스스로를 공격하는 꼴이 된다. 회복 중인 서버에 재시도 요청이 한꺼번에 몰리면 서버는 다시 죽는다. 이를 thundering herd 문제라 부른다.

해결책은 지수적 백오프와 jitter 두 가지다. 지수적 백오프는 첫 재시도를 1초 후, 두 번째를 2초 후, 세 번째를 4초 후로 점점 늦춰 서버가 회복할 시간을 벌여준다. jitter는 여기에 무작위성을 더하는 방법이다. 대기 시간에 무작위 값을 얹어 재시도 타이밍을 분산시키는데, 예컨대 기본 대기 시간이 1초라면 0에서 1초 사이의 값을 무작위로 더한다.

구체적인 백오프 계산식은 $\text{wait_time} = \min(\text{cap}, \text{base} * 2^{\text{attempt}}) + \text{random}(0, \text{base})$ 형태다. base는 1초, cap은 60초 정도로 잡고, attempt가 5를 넘으면 재시도를 멈추는 게 보통이다.

에이전트는 재시도 여부만 판단하는 데 그치지 않고 실제 대기 시간까지 지켜야 한다. 비동기 환경이면 `asyncio.sleep`을, 스레드 환경이면 `Thread.sleep`을 쓴다.

2.3 재시도 정책의 계층적 설계

재시도 정책은 도구 단위가 아니라 시스템 단위로 설계해야 한다. 도구마다 재시도 로직을 따로 두면 정책이 흩어져 관리하기 어려워진다.

좋은 구조는 세 계층으로 나뉜다. 가장 바깥쪽에는 에이전트 계층이 있어 도구를 호출하고 응답을 받는다. 그 안쪽 재시도 관리 계층은 도구 호출을 감싸 일시적 에러가 나면 재시도를 처리하고, 가장 안쪽 도구 계층이 실제 비즈니스 로직을 실행한다.

재시도 관리 계층은 도구 호출을 감싸는 래퍼로 구현한다. 파이썬이라면 데코레이터 패턴이 적합해서 `@retry_when(temporary_error,`

max_attempts=3, backoff_base=1) 같은 형태로 재시도 정책을 적용할 수 있다. 그러면 도구 개발자는 비즈니스 로직에만 집중하고 재시도 정책은 인프라가 맡는다.

2.4 복구 지점을 설계하는 법

에이전트가 긴 작업을 수행할 때 중간에 실패하면 처음부터 다시 시작해야 할 수 있다. 이를 방지하려면 복구 지점을 설계한다.

예를 들어 사용자에게 이메일을 보내는 작업이 있다고 하자. 이 작업은 이메일 주소 조회, 이메일 본문 생성, 이메일 발송 세 단계로 구성된다. 이메일 발송에서 실패하면 이미 조회한 이메일 주소는 유효하므로 처음부터 다시 조회할 필요가 없다.

복구 지점은 상태로 관리한다. 각 단계 완료 시 체크포인트를 저장하고, 실패 시 마지막 체크포인트부터 재개한다. 상태는 에이전트의 컨텍스트에 저장하거나 외부 저장소에 기록할 수 있다.

체크포인트를 설계할 때 고려할 점은 복원 비용이다. 상태 저장 자체가 비용이므로 저장 빈도와 저장소 사이에서 트레이드오프를 따져야 한다. 멍등성 확보도 중요하다. 복구 지점에서 다시 실행해도 결과가 같아야 하므로 도구 호출을 멍등하게 설계한다.

2.5 실패를 정보로 전환하는 채널

에이전트의 실패는 시스템 개선의 자료가 된다. 실패 패턴을 수집하면 어떤 도구가 어떤 에러로 실패하는지 알 수 있고, 시스템의 weak point를 찾을 수 있다.

실패 수집 구조에는 네 가지 정보가 필요하다. 에러 타입은 네트워크 에러인지 타임아웃인지 권한 에러인지를 구분하고, 컨텍스트는 어떤

파라미터로 호출했는지와 어느 단계에서 실패했는지를 남긴다. 재시도 횟수는 몇 번 시도했는지, 성공까지 걸린 시간은 얼마였는지를 보여주며, 마지막으로 추적 정보는 실패한 도구 호출의 전체 체인을 따라가 에이전트의 판단 과정을 분석하게 해준다.

이렇게 모은 정보는 두 가지로 쓸모가 있다. 하나는 에이전트의 행동 패턴 파악이다. 특정 도구에서 실패가 잦으면 그 도구의 스키마나 설명을 손봐야 할 신호다. 다른 하나는 시스템 전반의 건전성 모니터링으로, 재시도율이 갑자기 치솟으면 인프라 쪽 문제일 가능성을 의심해볼 수 있다.

2.6 정리

리트라이와 에러 복구 루프는 에이전트 시스템의 회복력을 결정한다. 에러를 세 가지로 구분하고 지수적 백오프와 jitter로 재시도 정책을 설계한 뒤 계층적으로 관리하면 시스템은 한층 견고해진다. 복구 지점을 마련해두면 긴 작업도 실패한 지점에서 다시 시작할 수 있고, 실패 정보를 꾸준히 모으면 시스템을 개선할 근거도 쌓인다.

3 Observability: 에이전트 내부에서 무슨 일이 일어나는지 알기

에이전트가 도구를 호출하고 결과를 처리하는 과정은 겉에서 보면 블랙박스나 다름없다. “왜 이 에이전트는 같은 실수를 반복하는가”, “어느 단계에서 병목이 생기는가” 같은 질문에 답하려면 에이전트의 행동을 눈에 보이게 만들어야 한다. 이것이 Observability다.

3.1 도구 호출 체인의 구조화된 로그

에이전트 로그를 단순 문자열로 남기면 정보 가치가 떨어진다. “도구 호출 시작”, “도구 호출 완료” 같은 로그만으로는 분석할 수 없다. 구조화된 형태로 남겨야 한다.

구조화된 로그가 갖춰야 할 핵심 필드는 다섯 가지다. timestamp는 호출 시점을 정확히 남기고, tool_name은 어떤 도구가 호출됐는지 식별한다. input에는 도구에 전달한 파라미터를, output에는 도구가 반환한 결과를 저장한다. duration은 호출부터 완료까지 걸린 시간을 잰다.

여기에 trace_id를 추가하면 좋다. 하나의 에이전트 실행을 구분하는 고유 ID를 만들어 모든 도구 호출 로그에 넣어두면, 특정 실행의 로그만 따로 모아 분석할 수 있다. parent_id도 유용하다. 에이전트가 다른 에이전트를 호출했을 때 부모-자식 관계를 추적하게 해준다.

로그에 파라미터와 결과를 남길 때는 민감한 정보를 제외해야 한다. 비밀번호, API 키, 개인 식별 정보는 마스킹 처리한다. 결과 크기가 크면 처음 N 글자만 저장하고 잘렸다는 표시를 남긴다.

3.2 스패 기반 추적

스패는 작업 하나를 나타내는 단위다. 도구 하나를 호출하는 것도 스패이고, 에이전트 전체 실행도 스패이다. 스패에는 시작 시간, 종료 시간, 작업 이름, 부모 스패 ID가 담긴다.

스패를 시각화하면 에이전트의 실행 흐름을 한눈에 볼 수 있다. 어떤 도구를 먼저 호출했는지, 각 도구가 얼마나 걸렸는지, 병렬로 실행된 도구와 순차적으로 실행된 도구가 무엇인지 파악할 수 있다.

병렬 도구 호출을 추적할 때는 부모 스패와 자식 스패의 관계를 명확히 해야 한다. 에이전트가 동시에 세 가지 도구를 호출하면 세 개의 자식 스패이 같은 부모 스패를 가진다. 추적 시스템에서 이를 올바르게 표시하면 병렬 실행 패턴을 분석할 수 있다.

스패에 태그를 붙이면 필터링할 수 있다. 예를 들어 에이전트의 단계별로 “planner”, “executor”, “reviewer” 태그를 붙이면 특정 단계의 성능만 따로 분석하기 쉬워진다. 에러가 발생한 스패에는 “error” 태그를 붙여 실패 패턴만 뽑아낼 수 있다.

3.3 핵심 측정 지표 설계

관찰 가능한 시스템에서는 측정 지표가 핵심이다. 에이전트 시스템에서 모니터링해야 할 핵심 지표는 다섯 가지다.

먼저 도구별 성공률이다. 전체 호출 중 성공한 호출의 비율을 도구마다 측정하는데, 특정 도구의 성공률이 낮으면 그 도구의 스키마나 구현을 점검해야 한다는 신호다. 다음은 리트라이 빈도로, 각 도구가 평균 몇 번 재시도되는지를 본다. 재시도가 많다면 일시적 에러든 지연이든 근본 원인을 찾아야 한다. 세 번째는 평균 실행 시간이다. 도구별로 호출부터 완료까지 걸리는 평균 시간을 추적하면, 특정 도구의 레이턴시가

급격히 오를 때 성능 문제의 신호로 잡아낼 수 있다. 네 번째는 에러 유형 분포다. 어떤 유형의 에러가 얼마나 발생하는지 살펴보면 원인을 좁힐 수 있다. 네트워크 에러가 대다수라면 인프라 문제일 가능성이 크고, 권한 에러가 늘고 있다면 인증 처리 로직을 의심해야 한다. 마지막은 비용 추적이다. 에이전트가 도구를 얼마나 호출하고 거기에 비용이 얼마나 드는지 측정한다.

지표의 이상을 감지하려면 임계값을 정해둬야 한다. 도구 성공률이 95% 아래로 떨어지면 알림을 보내는 식이다. 임계값은 경험적으로 정하되 점진적으로 조정해가는 편이 낫다.

3.4 에이전트 판단 과정 추적

도구 호출 로그만으로는 부족하다. 에이전트의 판단 과정까지 추적해야 왜 그런 행동을 했는지 이해할 수 있다.

판단 과정 추적은 에이전트의 사고 로그를 구조화된 형태로 남기는 작업이다. 각 판단 지점에서 어떤 정보를 참조했는지, 왜 그렇게 판단했는지를 기록한다. 이렇게 하면 에이전트의 설명 가능성이 높아지고, 디버깅할 때 의사결정 경로를 따라갈 수 있다.

다만 사고 로그를 전부 수집하면 비용이 커진다. 중요한 판단 지점에서만 선별적으로 남기는 편이 현실적이다. 예를 들어 에이전트가 도구를 선택하는 시점, 실패 후 재시도 전략을 바꾸는 시점, 사용자에게 질문을 에스컬레이션하는 시점에만 판단 과정을 기록하는 식이다.

3.5 대시보드 설계

수집한 데이터를 제대로 활용하려면 대시보드가 있어야 한다. 좋은 대시보드는 시스템 상태를 한눈에 파악하게 해준다.

기본 구성을 하나 제안하면 이렇다. 최상단에는 전체 시스템 건강 상태를 한눈에 볼 수 있는 요약 카드를 두고, 성공률·평균 응답 시간·활성 에이전트 수를 표시한다. 그 아래에는 도구별 성능 테이블을 배치해 각 도구의 호출 수, 성공률, 평균 응답 시간, P99 응답 시간을 열로 보여준다. 테이블 하단에는 최근 실패 로그를 시계열로 나열해, 어떤 도구에서 어떤 에러가 났는지 빠르게 확인할 수 있게 한다.

대시보드는 정적이기보다 동적이어야 한다. 특정 도구를 클릭하면 그 도구의 상세 분석으로 파고들 수 있어야 하고, 시간 범위를 조정하면 과거 데이터와 비교할 수 있어야 한다.

3.6 정리

Observability는 에이전트 시스템 운영의 기초다. 구조화된 로그로 도구 호출 체인을 추적하고, 스패스로 성능을 들여다보고, 핵심 지표로 시스템 건강 상태를 감시한다. 판단 과정을 추적하면 에이전트가 왜 그렇게 행동했는지 알 수 있고, 대시보드를 갖추면 운영 효율도 올라간다.

4 인간-에이전트 협업 모델: 자동화와 통제의 경계

에이전트를 프로덕션에 배치할 때 모든 것을 완전히 자동화하면 위험이 따른다. 판단이 틀릴 수 있고, 예상치 못한 상황에 스스로 대응하지 못할 수도 있다. 그렇다고 모든 단계에 인간이 개입하면 자동화의 의미가 사라진다. 이 장에서는 자동화와 인간 통제 사이에서 균형점을 찾는 방법을 다룬다.

4.1 Human-in-the-Loop의 세 가지 수준

Human-in-the-Loop는 하나의 형태로 뭉뚱그릴 수 없다. 상황마다 적합한 수준이 다르다.

첫 번째는 검토 수준이다. 에이전트가 작업을 먼저 실행하면 완료된 결과를 인간이 검토한다. 틀리면 수정하고 맞으면 승인한다. 빠른 반복이 필요한 창조적 작업에 어울린다. 두 번째는 승인 수준이다. 에이전트가 실행 계획을 세우고 인간이 승인한 뒤에야 실행이 시작된다. 에이전트가 먼저 움직이지 않으므로 위험한 조작을 사전에 막을 수 있다. 세 번째는 감시 수준이다. 에이전트가 자율적으로 실행하고 인간은 이상이 발견될 때만 개입한다. 정상적으로 돌아가는 동안에는 인간이 관여하지 않는다. 이미 충분히 검증된 작업에 적합하다.

어떤 수준을 선택할지는 작업의 위험도와 에이전트의 신뢰도에 달려 있다. 위험한 조작일수록 통제 수준을 높여야 하고, 새로운 도구일수록 검토나 승인 단계를 거쳐야 한다.

4.2 에이전트의 판단을 인간이 검토하는 인터페이스

검토 수준의 HITL을 구현하려면 에이전트의 판단 근거를 인간이 이해할 수 있게 제시해야 한다.

좋은 검토 인터페이스는 판단 근거를 세 가지로 나눠 보여준다. 첫째는 상황 설명이다. 에이전트가 지금 무엇을 하려는지 설명하는 부분으로 “사용자가 요청한 내용을 바탕으로 데이터베이스에서 사용자 정보를 조회하려 합니다” 같은 문장이 표시된다. 둘째는 판단 근거다. 왜 이 행동을 선택했는지 에이전트가 이유를 제시하는데 “사용자 ID가 유효하므로 조회 도구를 선택했습니다”처럼 구체적인 이유를 보여준다. 셋째는 예상 결과다. 이 행동을 실행하면 어떻게 될지 예측을 표시한다.

이 세 가지를 보여주면 인간이 빠르게 판단할 수 있다. 정보가 부족하면 추가로 질문하고, 판단이 틀리면 수정을 지시하며, 맞으면 승인을 클릭한다.

실제로는 에이전트가 판단 근거를 스스로 설명하기 어려운 경우가 많다. 이때는 판단 과정을 강제로 기록하게 하면 된다. 에이전트가 각 도구를 호출하기 전에 “이 도구를 선택한 이유” 필드를 반드시 채우도록 하는 식이다.

4.3 위험 작업의 승인 게이트 설계

승인 수준에서 특히 중요한 것은 위험 작업에 두는 승인 게이트다. 모든 작업에 인간 승인을 요구하면 자동화의 효과가 사라지므로, 위험한 조작에만 승인 게이트를 둔다.

위험도를 판단하는 기준은 네 가지다. 첫 번째는 되돌릴 수 있는 정도다. 실행 후 취소할 수 없으면 위험도가 높다. 데이터 삭제, 금전 거래, 공개 가능한 정보 전송이 여기 해당한다. 두 번째는 영향 범위다. 영향을 받는 사용자가 많을수록 위험도가 높다. 시스템 설정 변경이나 대규모 데이터

처리가 이런 경우다. 세 번째는 정보 민감도다. 개인 정보, 비밀 정보, 금융 정보를 다루면 위험도가 높다. 네 번째는 비가역성이다. 한 번 실행되면 다른 조작으로 되돌릴 수 없다면 위험도가 높다.

승인 게이트는 이 네 가지 기준을 점수화해서 설계한다. 점수가 임계치를 넘으면 승인을 요구하고 넘지 않으면 자동으로 실행한다. 임계값은 업무 특성마다 다르므로 처음에는 보수적으로 잡고, 운영하면서 점진적으로 조정해 나간다.

4.4 취소와 롤백 채널 명시

에이전트 시스템에서는 예상치 못한 상황을 반드시 대비해야 한다. 취소와 롤백 채널을 명시적으로 설계한다.

취소는 실행 중인 작업을 중단시키는 것이다. 에이전트가 긴 작업을 수행하는 도중 인간이 문제를 발견하면 그 자리에서 멈출 수 있어야 한다. 취소는 에이전트의 현재 작업 스레드를 중단하는 방식으로 구현한다. 취소 요청을 받으면 에이전트는 하던 작업을 마무리하지 않고 즉시 멈춘다.

롤백은 이미 실행된 조작을 원래 상태로 되돌리는 것이다. 도구를 설계할 때부터 롤백까지 함께 고려해야 한다. 예를 들어 데이터를 삭제하는 도구를 만들면 같은 데이터를 복원하는 롤백 도구도 함께 만든다. 롤백 가능 여부는 에이전트가 스스로 판단할 수 없으므로 시스템 수준에서 보장해야 한다.

취소와 롤백을 언제 발동할지도 설계 대상이다. 인간이 명시적으로 요청할 수도 있고, 시스템이 자동으로 판단할 수도 있다. 예를 들어 에이전트가 정해진 시간 안에 작업을 끝내지 못하면 자동으로 중지시킨다. 이렇게 하면 무한 반복이나 과도한 실행을 막을 수 있다.

4.5 에이전트의 자율성과 통제의 균형

에이전트 시스템 설계에서 가장 어려운 문제는 자율성과 통제의 균형이다. 너무 많이 통제하면 에이전트의 능력이 제한되고, 너무 많이 방임하면 위험해진다.

균형점을 찾는 원칙은 점진적 확대다. 새로운 에이전트를 Deploy할 때는 높은 수준의 통제에서 출발한다. 검토 수준으로 운영하면서 에이전트의 행동 패턴을 지켜본다. 안정적으로 동작한다고 판단되면 통제 수준을 조금씩 낮춘다. 승인 수준을 거쳐 최종적으로는 감시 수준까지 넓혀간다.

각 단계에서 메트릭을 수집한다. 에이전트의 성공률, 인간 개입 빈도, 위험 상황 발생 횟수를 추적한다. 이러한 메트릭이 일정 수준 이상으로 안정하면 다음 단계로 진행한다. 메트릭이 불안정하면 한 단계 되돌아가는 것도 고려한다.

4.6 정리

인간-에이전트 협업은 결국 자동화와 안전 사이의 균형이다. HITL의 세 가지 수준을 이해하고 작업 특성에 맞게 적용한다. 검토 인터페이스는 판단 근거를 명확히 제시하고, 승인 게이트는 위험 작업을 엄격하게 통제한다. 취소와 롤백 채널을 마련해 예상치 못한 상황에도 대응할 수 있게 한다. 에이전트의 자율성은 점진적으로 넓혀가면서 메트릭으로 안정성을 확인한다.